RELATÓRIO PARCIAL

VagaCerta

Encontre sua vaga. Sem perder tempo.

Integrantes

Lucas Sehn Klauck
Artur Zanoello
Vitor Benedett Caldas
Erico Campos Kempfer
Vinicius de Morais Franzen Cordeiro

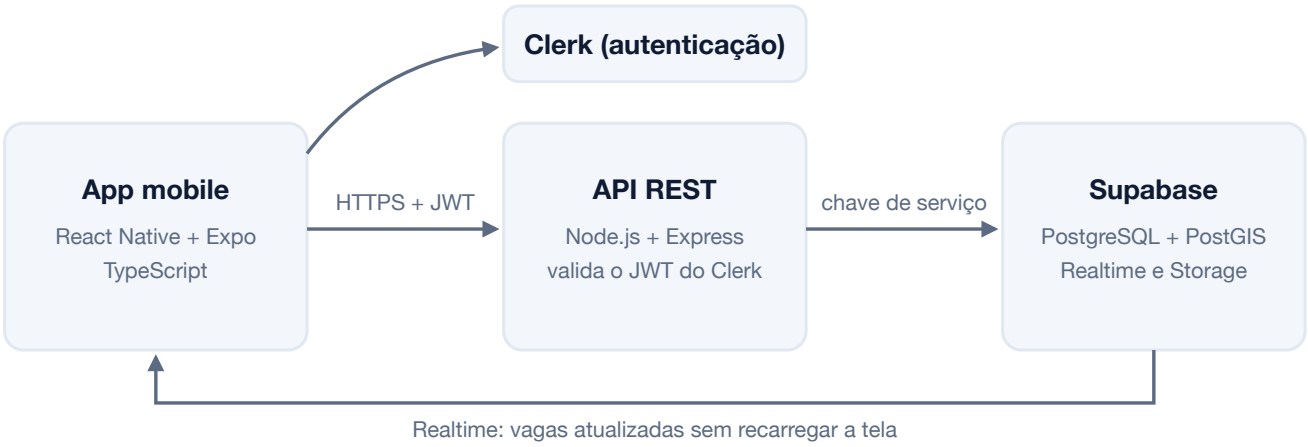
Data

3 de novembro de 2026

Objetivo e escopo

O VagaCerta é um aplicativo mobile para Android e iOS que ajuda motoristas a encontrar estacionamentos em áreas urbanas. A partir da localização atual ou de um destino pesquisado, ele reúne em um só lugar disponibilidade de vagas, preço, distância e horário de funcionamento, e abre o app de mapas para chegar ao local escolhido. Como não há sensores, a disponibilidade é simulada por uma rotina no backend.

Arquitetura e tecnologias



Camada	Tecnologias
App	TypeScript, React 19, React Native 0.88, Expo SDK 58, Expo Router
Recursos nativos	expo-location, expo-notifications, expo-audio, react-native-maps, expo-secure-store
Autenticação	Clerk: e-mail e senha, Google e Apple. O token é guardado com segurança no aparelho
Backend	Node.js, Express, helmet, cors e express-rate-limit
Dados	Supabase (PostgreSQL), PostGIS para distância, Realtime para vagas ao vivo

Como as partes se comunicam

O app faz login direto no Clerk e recebe um token. Cada chamada protegida à API leva esse token, que o Express valida antes de consultar o Supabase. A chave de serviço do banco fica só no backend. As mudanças de vagas chegam ao app direto pelo Realtime do Supabase, com a chave pública.

O Git usa uma branch por aluno, e cada PR é testado em `develop` antes de ir para `main`.

11 de 17
requisitos funcionais com entrega em branch

6 de 17
requisitos ainda não iniciados

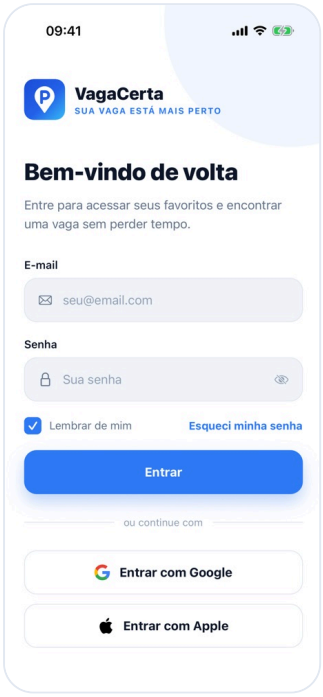
2 de 3
frentes já reunidas na develop. Falta a autenticação

O que está feito e funcionando

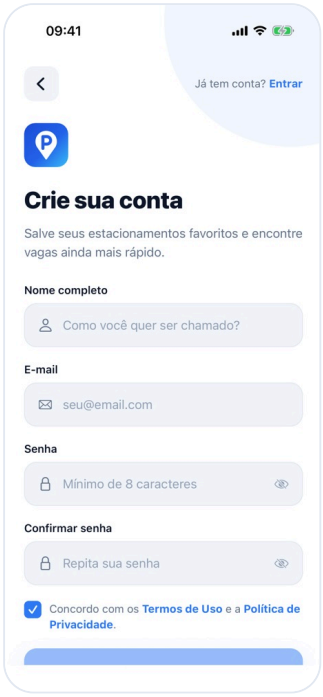
Lucas Sehn Klauck: autenticação, permissões e base do backend

dev-lucas · RF01, RF13, RF14, RF17

Login e criar conta ligados ao Clerk, com Google e Apple, e recuperação de senha. Lucas registrou criar conta e recuperar senha como testados. Permissões reais de localização, notificações e microfone. Base do backend com autenticação JWT e segurança.



Login
E-mail, senha, Google e Apple



Criar conta
Validação e verificação por código



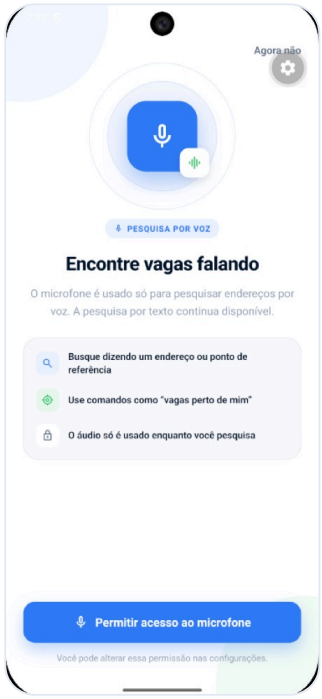
Esqueci minha senha
Código enviado por e-mail



Localização
Permissão real, sem travar se negar



Notificações
Permissão e preferências locais



Microfone
Permissão para a busca por voz

Linha de cima: iOS Simulator com Expo Go, em 03/11. Linha de baixo: prints de Lucas no emulador Android, em 29/10.

Artur Zanoello: Home e disponibilidade de vagas

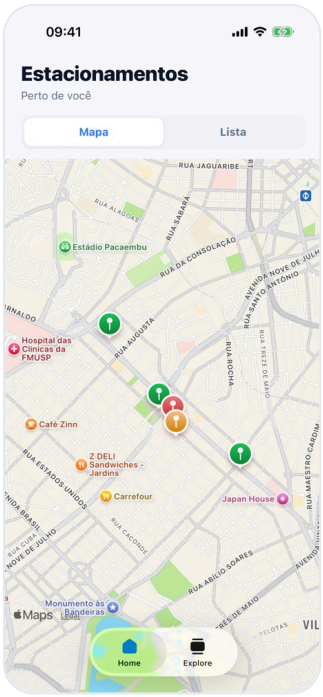
feature/artur-home, PR #7 · RF02, RF04, RF05, RF12

Mapa e lista com o mesmo conjunto de dados, ordenados por distância. Status por cor, vagas livres, rota `GET /parkings`, schema PostGIS com 18 estacionamentos de exemplo, simulação de vagas e assinatura Realtime.

Vitor Benedett Caldas: detalhes e Como chegar

feature/vitor-detalhes, PR #5 · RF06, RF08, RF09

Tela de detalhes com preço, vagas, horários da semana e destaque de Aberto agora. Rota `GET /parkings/:id`, tabela `parking_hours` e botões para Google Maps e Waze.



Home: mapa (Artur)
Cor do pino indica o status



Home: lista (Artur)
Distância, preço e vagas livres



Detalhes (Vitor)
Horários e Como chegar

iOS Simulator com Expo Go, em 03/11, com dados de exemplo do próprio app. O motivo está na próxima página.

Execução do backend e verificações

```
$ npm test                              parkingStatus.js ok · hours.js ok
GET /health                             200 {"status":"ok"}
GET /me          (sem token)            401 {"error":"Não autenticado"}
GET /parkings    (sem token)            401 {"error":"Não autenticado"}
tsc --noEmit                            develop ok · dev-lucas ok
expo lint                               develop ok · dev-lucas 1 erro
```

O que falta ou está em andamento

Em andamento: integração

- **Autenticação fora da `develop`.** O trabalho de `dev-lucas` ainda não tem PR. Por isso, o login da `develop` é só visual. O merge tem conflitos em `package.json`, `package-lock.json` e `use-user-location.ts`.
- **Home e Detalhes sem dados reais.** A API exige token, mas o app ainda chama a rota sem ele e recebe 401. Até a integração, as telas só funcionam com os dados de exemplo.
- **Ainda não validados de ponta a ponta:** vagas mudando ao vivo pelo Realtime, fotos do Storage e a abertura do Google Maps e do Waze em aparelho real.
- **Ajustes técnicos:** corrigir o `package-lock.json` da `develop`, pois `npm ci` falha, e um erro de lint em `dev-lucas`.

Não iniciado

Funcionalidade	Requisitos
Busca por endereço ou região	RF03
Filtros de busca	RF07
Pesquisa por voz	RF16. Pode exigir development build, pois o Expo Go não inclui o módulo de fala
Favoritos	RF10, RF11. Há só um espaço reservado na tela de detalhes
Perfil e logout	RF15. A aba Explore ainda é a do template do Expo

Dificuldades e próximos passos

Obstáculos encontrados

- **Merge indevido na `main`:** o PR #4 foi para a branch errada. Vitor reverteu no PR #6 e refez pela `develop` no PR #5. Agora os PRs são testados na `develop` antes da `main`.
- **Dependência entre tarefas:** as telas dependem da base do backend, da autenticação e do schema, feitos em paralelo. Um efeito disso é que o hook de localização foi criado nas branches de Lucas e de Artur.
- **Limites do Expo Go:** ele não mostra a splash personalizada e não inclui o módulo de reconhecimento de fala.
- **Atualização do Expo:** o projeto passou do SDK 57 para o 58 no meio do desenvolvimento.

Plano até a entrega final

1. Abrir o PR de `dev-lucas` para `develop` e resolver os conflitos.
2. Usar o cliente autenticado nas chamadas à API e testar login, Home e Detalhes com dados reais.
3. Validar Realtime, fotos e Como chegar em aparelho real.
4. Implementar busca, filtros e voz, definindo se será preciso development build.
5. Implementar favoritos e perfil, e remover a aba do template.
6. Fazer o teste completo na `develop` e abrir o PR final para `main`.